

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 2 - Amrut Anand - Roll: 24011101003

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset Description

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

The dataset images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

GitHub repository: https://github.com/the-future-codemaster/dl_lab/tree/main/Experiment_2

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Task 1: Dataset Dimensions

Dataset Split	Features Shape	Labels Shape
Training Set	(60000, 28, 28)	(60000,)
Testing Set	(10000, 28, 28)	(10000,)

Task 2: Tensor Shapes Before and After Preprocessing

Training Tensor	Original Shape	Preprocessed Shape
Features (x_{train})	(60000, 28, 28)	(60000, 784)
Labels (y_{train})	(60000,)	(60000, 10)

Best Hyperparameters

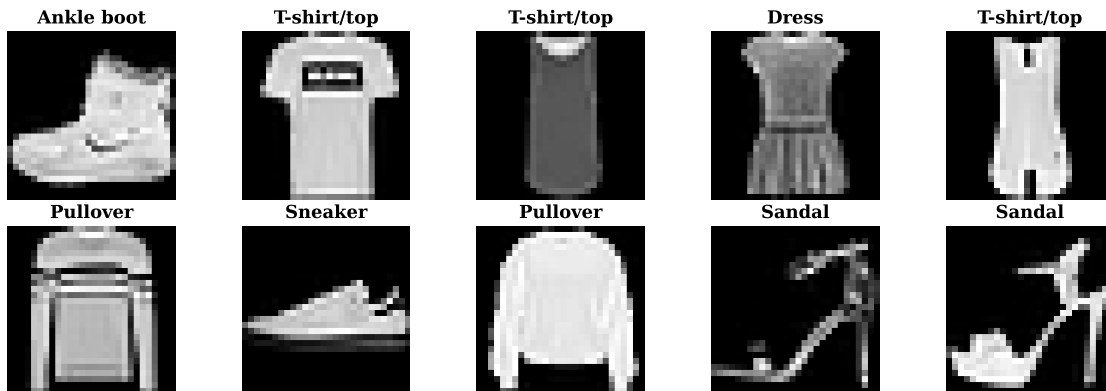
Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	adam
Activation Function	relu
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8899
Testing Accuracy	0.8813

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8875	0.8813
Precision	0.8888	0.8847
Recall	0.8875	0.8813
F1-score	0.8876	0.8813
Training Time	116.56 s	6323.77 s

8. Plots & Interpretations

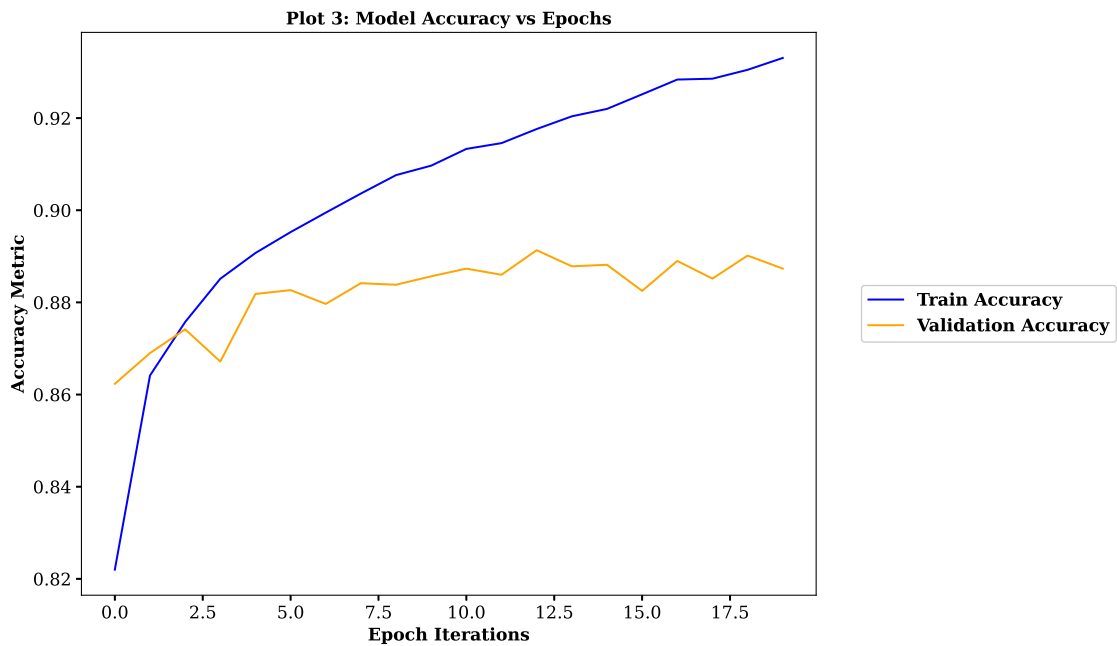
Plot 1: Ten Sample Apparel Images



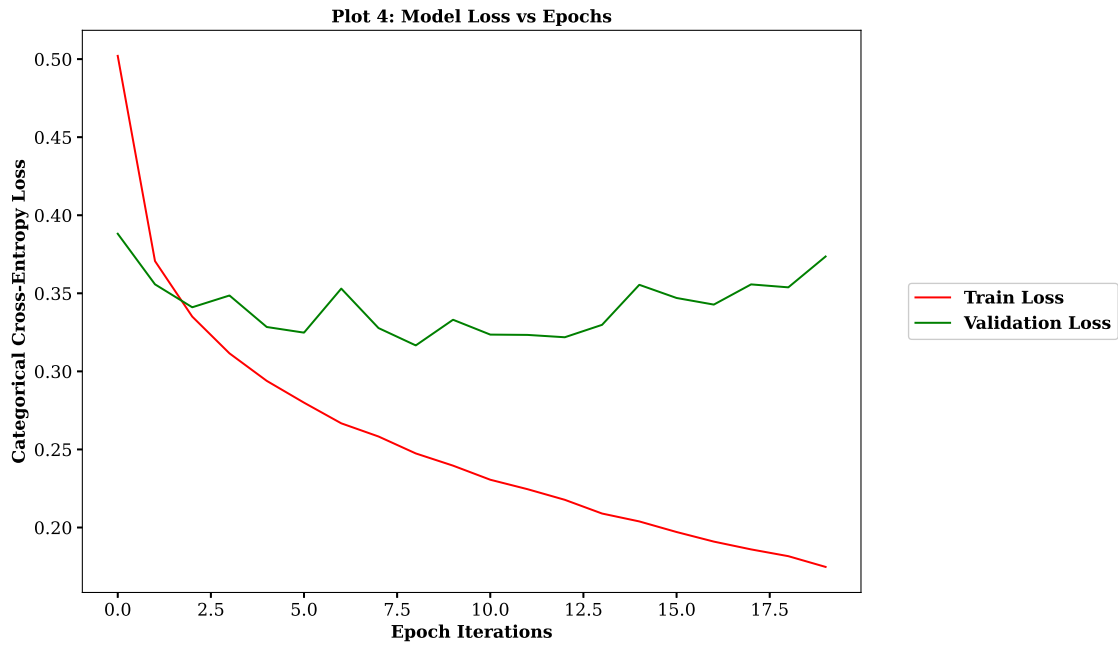
Interpretation: We can observe that some items like pullovers, coats and shirts share overlapping silhouette features. There is also observable intra-class structural diversity and inter-class geometric similarities.



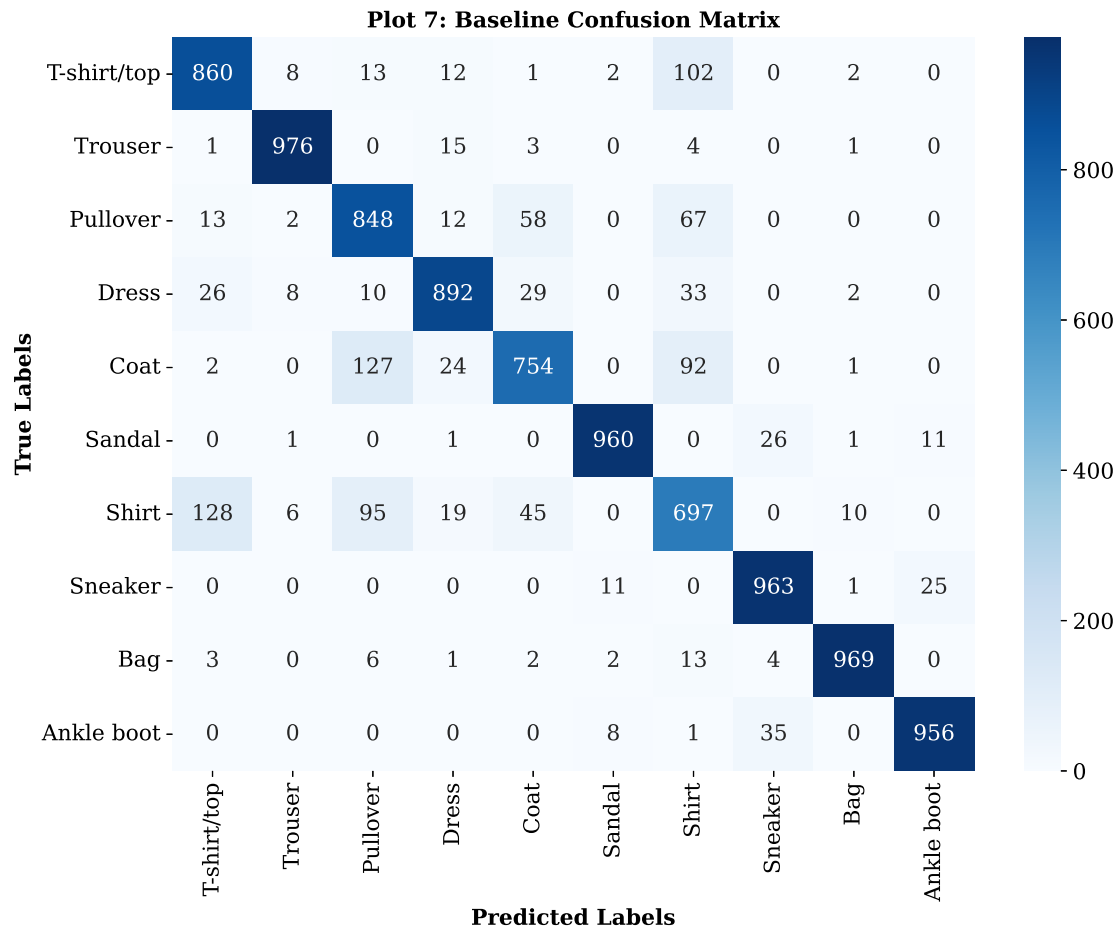
Interpretation: This is a uniform class distribution with exactly 6000 samples for each of the 10 categories. We do not need class-weight adjustments as the symmetry avoids majority-class bias during gradient descent.



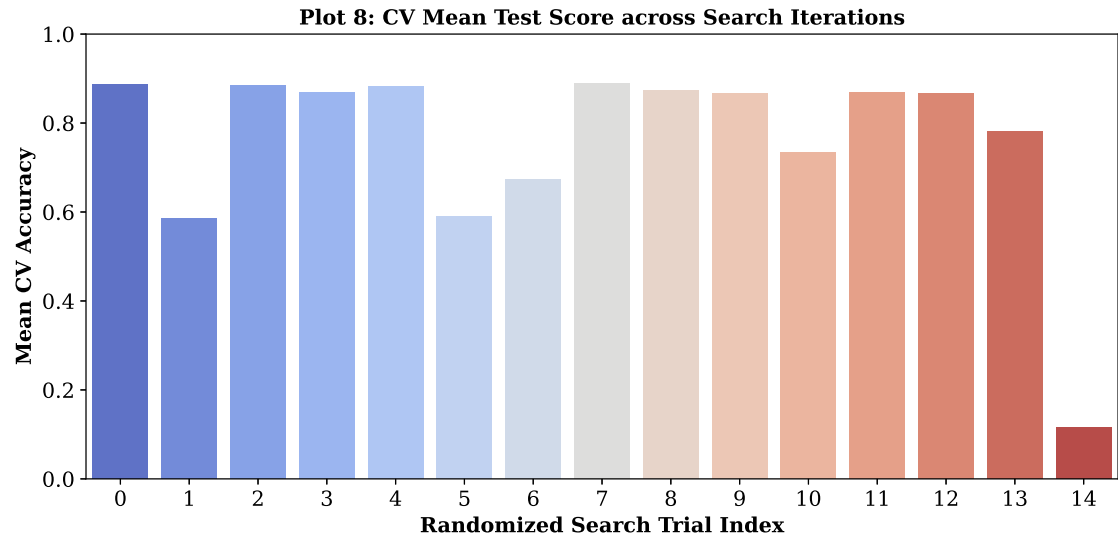
Interpretation: The training accuracy increases monotonically till near 0.93. On the other hand validation accuracy saturates quickly and instead starts to oscillate around 0.89. This divergence conveys model overfitting where it is starting to memorising training-specific noise.



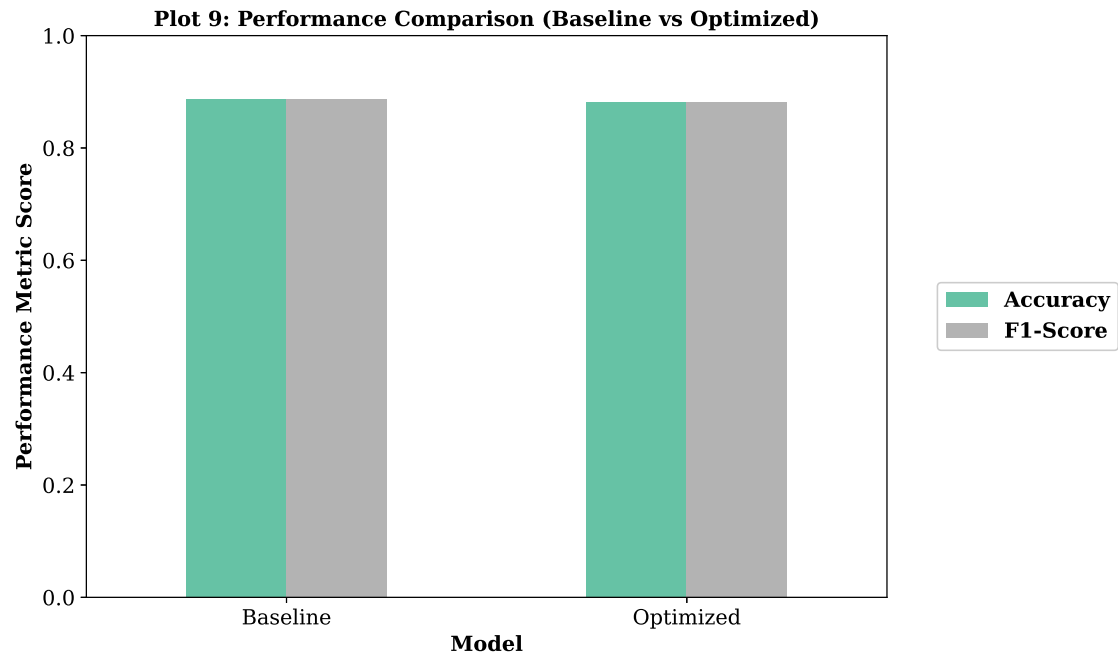
Interpretation: The model is learning from the training data and improving because the categorical cross-entropy shows a continuous decay in training error across all 20 epochs. The validation loss reaches the global minimum early in the training cycle. Then it fluctuates before gradual increasing showing overfitting.



Interpretation: The diagonal validates the model’s predictive capability. However the few off-diagonal clusters show the semantic ambiguities present within the upper-body garments. Eg: 'Shirt' is misclassified as 'T-shirt/top' 128 times and 'Pullover' 95 times. This shows the limitation in identifying the fine-grained textural boundaries.



Interpretation: This graph shows the importance of choosing the hyperparameters correctly to reduce generalisation error. Across different combinations, some show very good CV accuracies while some are much worse showing strong dependency.



Interpretation: We observe a decrease in the optimised model from the baseline model where accuracy goes from 88.75% to 88.13% and F1-Score from 88.76% to 88.13%. This is because of overfitting during training.

9. Conclusion

We successfully implemented and evaluated a Multi-Layer Perceptron for the Fashion-MNIST image classification dataset. We flattened a 28×28 matrix and normalised it into 784-dimensional vectors. Though the baseline model had generalised well, it had some ambiguities among upper-body garments which had similar structures.

Hyperparameter Initialisation was crucial for the convergence of the 5-fold cross-validated Randomized-SearchCV. The configuration of a single 256-neuron layer, Adam optimizer, learning rate 0.001, and 0.2 dropout had a generalisation gap on unseen testing data as opposed to the default model. Therefore for this dataset we can say that the multi-layered hierarchical feature extraction i.e. the 128-to-64 neuron baseline model is more effective than the shallow network although high-capacity, for identifying the spatial differences in the apparel data.

10. Discussion

1. Which optimization method was used?

We used RandomizedSearchCV: a 5-fold cross-validated Randomized Search. GridSearchCV which tries every single possible combination takes too much time and computational resources. This method instead picks different combinations of hyperparameters to test from specific distributions. It saves a lot of time while still finding really good settings.

2. Which hyperparameters were selected?

The best model used 1 hidden layer with 256 neurons. To prevent overfitting, a dropout rate of 0.2 was used along with activation function as relu. The optimiser was Adam along with a learning rate of 0.001. The data was processed in batch sizes of 16 for 20 epochs.

3. Did optimization improve performance?

No. Even though our settings had a high cross-validation accuracy of 88.99%, when it was tested on completely unseen data, the accuracy dropped to 88.13%. This was lower than the baseline model which scored 88.75% meaning the model overfitted a little to the training data and couldn't get a similar score on test.

4. Which hyperparameter had the greatest impact?

The greatest impact was decided by the optimiser and learning rate specifically - the Adam optimizer and an learning rate of 0.001. Adam adapts its step size while learning which keeps the training smooth. It also stops loss from fluctuating like it does when we have the standard SGD.

5. Compare Grid Search and Randomized Search.

Grid search is a brute-force approach. It tries every combination of settings which guarantees the best combination but it is computationally heavy. On the other hand, Randomized Search takes random combinations of hyperparameters to test from specific distributions. It takes a lot lesser time and finds almost ideal settings.

6. Recommend the best MLP configuration.

The original baseline model with the first hidden layer of 128 neurons and 2nd hidden layer of 64 neurons is recommended. Having more layers helped the model learn the shapes of the clothing better. We could also predict that a two-layer model having ReLU and a dropout for safety of around 0.2 along with the Adam optimizer at a 0.001 learning rate could give ideal results.

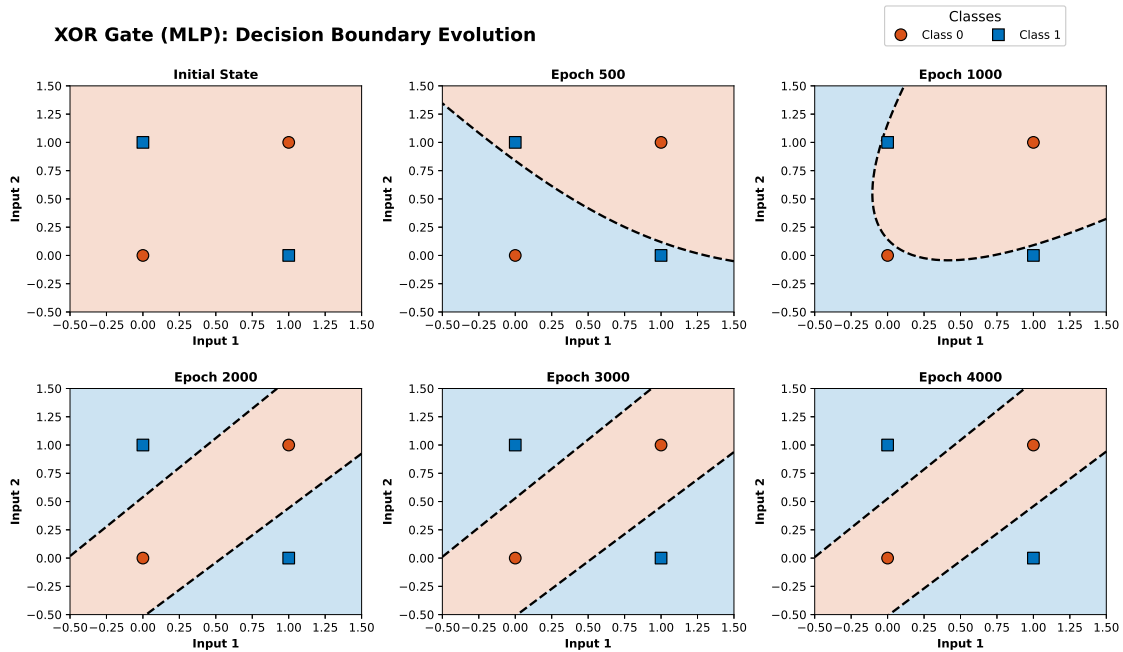
11. Additional Tasks

Task: Multi-Layer Perceptron for XOR Gate

XOR Gate Training History (Weights & Biases)

Epoch	Hidden Layer (W_1, b_1)	Output Layer (W_2, b_2)
500	W_1 : $[-0.339, 0.871]$, $[0.531, -0.217]$ b_1 : $[[0.173, -0.001]]$	W_2 : $[-0.476]$, $[-0.341]$ b_2 : $[[0.464]]$
1000	W_1 : $[-0.912, 1.377]$, $[1.209, -0.822]$ b_1 : $[[0.738, 0.533]]$	W_2 : $[-0.821]$, $[-0.724]$ b_2 : $[[1.022]]$
2000	W_1 : $[-5.157, 5.330]$, $[5.346, -5.113]$ b_1 : $[[2.649, 2.616]]$	W_2 : $[-5.521]$, $[-5.524]$ b_2 : $[[8.068]]$
3000	W_1 : $[-5.710, 5.885]$, $[5.898, -5.677]$ b_1 : $[[2.912, 2.889]]$	W_2 : $[-6.543]$, $[-6.546]$ b_2 : $[[9.609]]$
4000	W_1 : $[-5.962, 6.139]$, $[6.151, -5.933]$ b_1 : $[[3.033, 3.013]]$	W_2 : $[-7.056]$, $[-7.058]$ b_2 : $[[10.381]]$

Note: MLP Converged correctly.



Interpretation: We get an output of 1 when both the inputs are different i.e. $[0,1]$ or $[1,0]$ from a XOR Gate. A Single-Layer Perceptron cannot separate the positive from the negative class because a straight line cannot separate a diagonal cross. Therefore it fails. A Multi-Layer Perceptron used backpropagation and hidden layers to change the feature space. It changes the decision boundary to curve around the inputs dynamically and solve the non-linear problem.

12. References

1. Goodfellow et al., *Deep Learning*.

2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.